DTT
APPS WEB GAMES

SINCE 2010

Level Select

Documentation | 07-06-22

Table of Contents

1. Get started quickly 3

2. Introduction 4

3. Set-Up 5

4. Editor 6

5. API 7

MyClass 7

6. Known Limitations 8

7. Support and feedback 9

1. Get started quickly

1. Place the “Level Select <orientation>” prefab from the “Assets/DTT/Level Select/Demo/Prefabs” folder into your scene.
2. You can now create new Scriptable Object Compositions to determine how an area in your level select environment should look. This can be done from the create menu under “DTT/Level Select/Composition”.
3. In the Composition you can configure the various layouts that should be used in your level select. Make sure the amount of Level Buttons in your layout matches the amount configured on the Level Select component.
4. Afterwards, make sure to place your new composition on your Level Select gameobject.
5. Now you only have to hook up your level database to the Level Select. This can be done by making a new class that inherits from the LevelRetriever component. This will force you to implement several methods. When this is done, reference this component in the Level Select component.

2. Introduction

The DTT Level Select asset aims to provide you with a customizable environment in which your users can select the levels of your game. We support different worlds to visualize in the selection environment, so you can add variety to your level select.

Each level button has features, you might expect in a level select:

- Levels can be locked based on whether the user has unlocked them yet.
- Can display a score in form of stars.
- A visual highlight for the current level the player should play to progress.

3. Set-Up

Level Retriever

The first step to making use of the Level Select asset is to hook up your level database to the Level Select environment. This can be done by inheriting from the LevelRetriever class. You can check the demo example LevelRetrieverDemo in "Assets/DTT/Level Select/Demo". From this class, you have to implement two methods: one for retrieving a level and another for retrieving the total amount of levels. This data can then be returned by using the callback method. It is implemented in this way, so it can support operations from an API as well.
If all the levels are saved locally, you can just immediately return them.

After the LevelRetriever has been implemented you can place it in your scene and reference it on the LevelSelect component.

Level Layout

A LevelSelectLayout can be used to define how each segment of your level select environment will look. The environment will be generated by placing these layouts in succession next to each other based on some configuration will talk about in the Composition section.
You can take the demo layouts as an example for how to make each one look. To create your own layout, I suggest making a duplicate of one of the demo layouts and adjusting that to your needs. It is important that the prefab contains the LevelSelectLayout script and the LevelButtons placed inside are ordered from high to low, as that will be used to connect the levels between each other.

Composition

A composition is a scriptable object that can be used to define an 'area', 'world', or other place within your Level Select environment. This can be used to give more variety to your environment and indicate the different areas of your game that the levels will be themed around.

Each composition requires an entry and exit LevelSelectLayout so it can transition between the different compositions. The Centre Layouts can be used to determine how all the layouts in between the first and the last layout will look. You can add as many variants here as you want. By default, the layouts will be picked randomly.

4. API

LevelSelect

Property Name	Type	Description
ButtonPressed	Action<LevelButton>	Called when a LevelButton is pressed, passing the pressed LevelButton

LevelSelectLayout

Property Name	Type	Description
LevelButtons	ReadOnlyCollection<LevelButton>	The LevelButtons that are present in this layout

LevelRetriever

Method name	Return Type	Parameters	Description
Retrieve	void	int from int to Action<LevelData[ ]> callback	Retrieves all the level data for the level range provided
RetrieveLevelCount	void	Action<int> callback	Retrieves the total level count for the game. Can be used to determine where the game should stop.

5. Known Limitations

- Levels can only be connected straight-forward, no branching.
- Each level layout needs to contain the same amount of Level Buttons.
- Each layout needs to be the same size.

6. Support and feedback

If you have any questions regarding the use of this asset, we are happy to help you out. Always feel free to contact us at:

unity-support@d-tt.nl

(We typically respond within 1-2 business days)

We are actively developing this asset, with many future updates and extensions already planned. We are eager to include feedback from our users in future updates, be they 'quality of life' improvements, new features, bug fixes or anything else that can help you improve your experience with this asset. You can reach us at the email above.

Reviews and ratings are very much appreciated as they help us raise awareness and to improve our assets.

DTT stands for Doing Things Together

DTT is an app, web and game development agency based in the centre of Amsterdam. Established in 2010, DTT has over a decade of experience in mobile, game, and web based technology.

Our game department primarily works in Unity where we put significant emphasis on the development of internal packages, allowing us to efficiently reuse code between projects. To support the Unity community, we are publishing a selection of our internal packages on the Asset Store, including this one.

More information about DTT (including our clients, projects and vacancies) can be found here:

https://www.d-tt.nl/en/